EQUIPO OMERO · PROYECTO DE INFORMÁTICA I

Cómo trabajamos en GitHub

De la issue al merge en 8 pasos, y cómo escribir los commits.

La idea en 30 segundos

Cada tarea es una **issue** del Project. Tú solo te ocupas de **tus issues**: desde la propia issue creas la rama, trabajas, abres una Pull Request y, al hacer merge, la issue se cierra sola y la tarjeta pasa a *Done*.

1. **Issue** — Te la asignan en el Project
2. **Rama** — Se crea desde la issue
3. **Commits** — Trabajas en tu rama
4. **Pull Request** — Hacia `develop`
5. ✓ **Merge** — Issue cerrada, tarjeta en Done

Las ramas del repo

`main`

Versión de entrega. Solo cambia en cada entrega. Protegida: solo la actualiza JP.

`develop`

Donde se junta el trabajo de todos. Protegida: solo entra código por Pull Request.

`12-nombre-tarea`

Tu rama de trabajo. Una por issue. Se crea desde la issue y se borra al hacer merge.

✕ **Nunca trabajes directamente en `develop` ni en `main`.** GitHub no te dejará hacer push.

Antes de empezar (solo una vez)

A — Acepta las invitaciones

Te llegará un email para unirte al **repositorio** y otro para el **Project**. Acepta los dos. Si no los encuentras, mira en `github.com/notifications`.

B — Dile a Git quién eres

Usa el mismo email que en tu cuenta de GitHub, así tus commits aparecen con tu nombre.

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu-email@ejemplo.com"
```

C — Clona el repo y ábrelo en tu IDE

```
git clone https://github.com/JP-Aceves/OMERO-HOSPITAL.git
cd OMERO-HOSPITAL
```

Ábrela en PyCharm o VS Code. Todos los comandos se ejecutan **dentro de esa carpeta**.

01 Cada vez que tengas una tarea

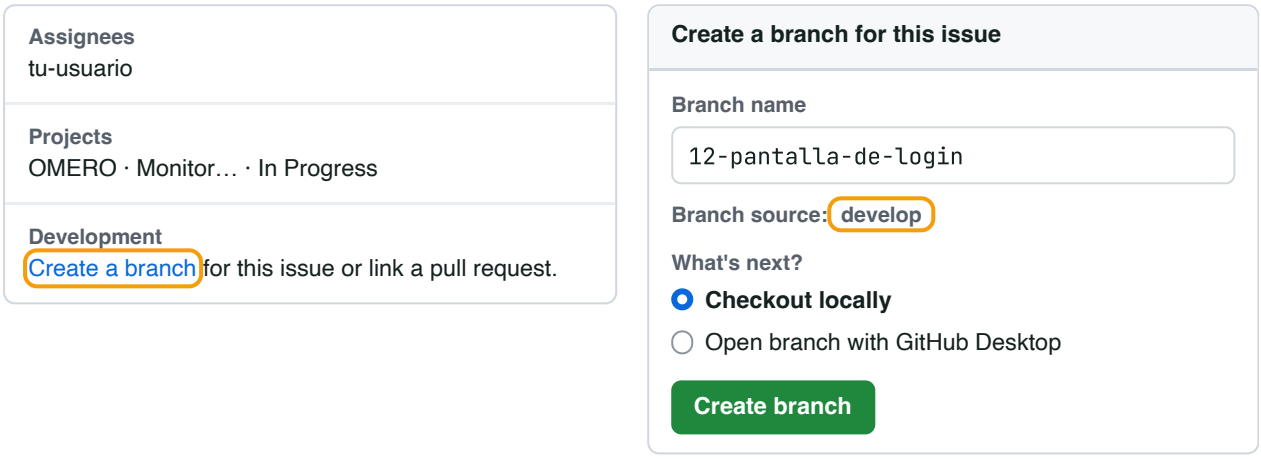
Repite estos 8 pasos con cada issue. Las etiquetas indican dónde se hace cada paso: GITHUB en la web y TERMINAL en la terminal de tu IDE.

1 Abre tu issue GITHUB

Entra al Project del equipo, pestaña **Mis tareas**, y abre la issue que te toca. Cámbiale el *Status* a **In Progress** para que el resto sepa que estás con ella.

2 Crea la rama desde la issue GITHUB

En la columna derecha de la issue, en el apartado **Development**, pulsa **Create a branch**.



Deja el **nombre** que propone GitHub (número de issue + título). Comprueba que el origen de la rama sea `develop`. Si no lo es, pulsa *Change branch source* y elige `develop`. Marca **Checkout locally** y pulsa **Create branch**.

3 Cámbiate a tu rama TERMINAL

GitHub te mostrará dos comandos para copiar. Pégalos en la terminal:

```
git fetch origin
git checkout 12-pantalla-de-login
```

A partir de aquí, todo lo que hagas queda en tu rama. Si quieres comprobar en qué rama estás: `git branch` (la tuya sale marcada con `*`).

4 Trabaja y haz commits TERMINAL

Programa con normalidad. Cada vez que termines un trozo con sentido (algo que funciona), guárdalo con un commit. El formato del mensaje se explica en la sección **02**.

```
git add -A
git commit -m "feat(auth): añade pantalla de login"
```

Es mejor hacer varios commits pequeños que uno enorme al final.

5 Sube tu rama TERMINAL

```
git push
```

Puedes hacer `git push` tantas veces como quieras mientras trabajas. Así tu trabajo queda guardado en GitHub aunque todavía no esté terminado.

---

6 Abre la Pull Request GITHUB

Cuando la tarea esté terminada y la app funcione, entra al repo. Verás un aviso amarillo con el botón **Compare & pull request**. Si no aparece, ve a la pestaña *Pull requests* → *New pull request* y elige tu rama.



- **base** tiene que ser `develop`, nunca `main`.
- El **título** sigue el mismo formato que los commits.
- En la **descripción**, escribe `Closes #12` con el número de tu issue. Así la issue se cierra sola al hacer merge.

---

7 Haz merge GITHUB

Si GitHub no marca conflictos, pulsa **Merge pull request** → **Confirm merge**. No hace falta que nadie te apruebe la PR. Al terminar:

- La issue se cierra y la tarjeta pasa a **Done** automáticamente.
- La rama se borra sola en GitHub.

> Si aparece **"This branch has conflicts"**, mira *Problemas frecuentes* en la página 5.

---

8 Vuelve a develop y limpia TERMINAL

```
git checkout develop
git pull
git branch -d 12-pantalla-de-login
```

Ya tienes en tu ordenador lo último de todos. Para la siguiente tarea, vuelve al paso 1.

02 Cómo escribir los commits

Usamos **Conventional Commits**: cada mensaje empieza por un tipo que dice qué clase de cambio es. Así el historial se lee de un vistazo y es fácil encontrar cosas.

`feat(auth): añade pantalla de login`

tipo (obligatorio) · ámbito (opcional) · descripción

TIPO	CUÁNDO USARLO	EJEMPLO
`feat`	Funcionalidad nueva	`feat(alertas): añade semáforo por umbrales`
`fix`	Arreglas un error	`fix(sensores): corrige lectura negativa del HC-SR04`
`docs`	Solo documentación (README, comentarios)	`docs: añade pasos de instalación al README`
`style`	Formato, espacios o nombres, sin cambiar lo que hace el código	`style: aplica PEP 8 en simulador.py`
`refactor`	Reorganizas el código sin cambiar su comportamiento	`refactor(datos): separa lectura y escritura de JSON`
`test`	Añades o arreglas pruebas	`test(prediccion): añade tests de la regresión lineal`
`perf`	Mejoras el rendimiento	`perf(graficas): reduce puntos del histórico`
`build`	Dependencias o empaquetado	`build: añade PySide6 a requirements.txt`
`chore`	Mantenimiento que no toca la app: configuración, .gitignore, limpieza	`chore: añade .gitignore para Python`

Reglas

- ✓ Todo en **minúscula** y **sin punto final**.
- ✓ La descripción completa la frase *"Este commit..."*: *añade*, *corrige*, *elimina*.
- ✓ **Máximo unos 70 caracteres.**
- ✓ **Un cambio por commit.** Si necesitas un "y", probablemente son dos commits.

Ámbitos del proyecto

Opcionales, pero ayudan a saber qué parte se ha tocado:

`auth` · `dashboard` · `sensores` · `alertas` · `datos` · `graficas` · `prediccion` · `actuadores` · `ui` · `usuarios`

Título de la PR

Mismo formato, resumiendo la tarea entera:
`feat(auth): pantalla de login con roles`

✓ Así sí

```
feat(dashboard): añade vista del técnico
fix(auth): impide login con campos vacíos
docs: documenta formato del JSON
```

✗ Así no

```
cambios
arreglado lo del login y más cosas
Feat: Añadí El Dashboard.
```

03 Reglas del equipo

- ✓ **Una issue = una rama = una PR.**
- ✓ Las PR **siempre** van a `develop`.
- ✓ Antes de abrir la PR, comprueba que la app arranca y que pasan los tests.
- ✓ Avisa en el grupo si vas a tocar un archivo que también usan otros.
- ✓ Si una tarea no tiene issue, pide que se cree antes de empezar.
- ✗ Programar en `develop` o en `main`.
- ✗ Abrir PRs hacia `main`. Solo JP pasa `develop` → `main` en cada entrega.
- ✗ Dejar una rama abierta muchos días: mejor tareas pequeñas.
- ✗ Subir archivos de configuración del IDE, `.venv` o contraseñas.

04 Problemas frecuentes

"This branch has conflicts" o mi rama se ha quedado atrás

Alguien ha cambiado lo mismo que tú en `develop`. Trae sus cambios a tu rama:

```
git pull origin develop
```

Tu IDE marcará los archivos en conflicto. Elige qué versión se queda (o combina las dos), guarda y termina con:

```
git add -A
git commit -m "fix: resuelve conflictos con develop"
git push
```

He programado en develop sin darme cuenta (y aún no he hecho commit)

```
git stash                          # guarda tus cambios aparte
git checkout 12-pantalla-de-login  # cámbiate a tu rama
git stash pop                      # recupera tus cambios ahí
```

Si ya habías hecho commit en `develop`, no hagas nada más y avisa a JP.

"push declined" o "protected branch"

Estás intentando subir a `develop` o a `main`. Comprueba tu rama con `git branch` y cámbiate a la tuya.

"The current branch has no upstream branch"

Pasa si creaste la rama en la terminal en vez de desde la issue. Súbela así:

```
git push -u origin HEAD
```

Lo de develop no aparece en mi ordenador

```
git checkout develop
git pull
```

Chuleta

Para tener a mano.

1 · En la issue GITHUB

Status → **In Progress**
Development → **Create a branch**
Source: **develop** · **Checkout locally**

2 · Cambiarte a tu rama

```
git fetch origin
git checkout 12-nombre-tarea
```

3 · Guardar trabajo

```
git add -A
git commit -m "feat: añade X"
git push
```

4 · PR y merge GITHUB

Compare & pull request
base: **develop** · descripción: **Closes #12**
Merge pull request → Confirm

5 · Volver y limpiar

```
git checkout develop
git pull
git branch -d 12-nombre-tarea
```

Tipos de commit

`feat` nueva función · `fix` error
`docs` documentación · `style` formato
`refactor` reorganizar · `test` pruebas
`perf` rendimiento · `build` dependencias
`chore` mantenimiento

¿En qué rama estoy? `git branch` · **¿Qué he cambiado?** `git status` · **¿Dudas?** Pregunta en el grupo antes de forzar nada.